1. Least Square Filtering (최소제곱법 기반)

개념

- **최소제곱법(Least Squares Method)** 은 주어진 데이터 포인트와, 이를 설명하는 어떤 함수 f(x) 간의 오차(잔차, residual) 제곱합이 최소가 되도록 f(x)를 찾는 방법.
- 1차(직선), 2차(포물선) 등 다항식으로 근사해도 되고, 특수한 형태(예: 지수함수 등)로 근사해도 됨.
- 예) 선형회귀(Linear Regression)
  -> 많은 (x, y) 데이터가 있을 때, "직선" 하나로 가장 잘 맞추는 문제.
  역시 최소제곱법을 이용함.

1.2 수식 예시 (다항식 근사)

데이터가 (x_i, y_i) ($i = 1, 2, \ldots, N$) 로 주어지고, 이 데이터를 n차 다항식

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

으로 근사한다고 가정함. 최소제곱법에 따라, **오차 제곱합** S 는

$$S(a_0, a_1, \ldots, a_n) = \sum_{i=1}^{N} \big(y_i - f(x_i)\big)^2$$

이 되고, 이것을 $a_0, a_1, \ldots, a_n$ 에 대해 미분 = 0이 되도록 풀면(정규방정식),

$$\begin{pmatrix} N & \sum x_i & \sum x_i^2 & \ldots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \ldots & \sum x_i^{n+1} \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 & \ldots & \sum x_i^{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \sum x_i^{n+2} & \ldots & \sum x_i^{2n} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \\ \vdots \\ \sum x_i^n y_i \end{pmatrix}.$$

이 선형방정식을 풀면 근사 다항식의 계수 $a_0, a_1, \ldots, a_n$를 구할 수 있음.

- 실제 프로그래밍에서는 보통 선형대수 패키지(예: numpy.linalg.lstsq) 등을 사용하여 쉽게 계산함.

구현 예시 (Python)

```python
import numpy as np

# x, y가 1차원 numpy array라고 가정
# n차 다항식을 맞추고 싶다면
def least_squares_polyfit(x, y, degree=2):
    # Vandermonde 행렬(각 열이 [1, x, x^2, ...] 형태)
    A = np.vander(x, degree + 1)
    # 최소제곱 해를 구함
    coeffs, residuals, rank, s = np.linalg.lstsq(A, y, rcond=None)
    return coeffs  # [a_n, a_(n-1), ..., a_0] 순서
```

2. Bezier (베지어) 곡선 기반

개념

- 베지어 곡선(Bezier curve) 은 그래픽 분야에서 널리 쓰이는 곡선 표현 방식임.
- 주로 제어점(control points) 을 이용하여 곡선을 부드럽게 연결함.
- 2차(Quadratic) 혹은 3차(Cubic) 베지어 곡선이 일반적임.
- 스무딩이 필요한 데이터가 있을 때, 데이터 전체에 대해 베지어 곡선을 직접 맞추기보다는, 구간별로 control points를 잡아 곡선을 이어 붙이거나, 스플라인(spline) 기법을 혼합해서 쓰기도 함.

2.2 수식 (3차 베지어 곡선 예시)

3차 베지어 곡선의 정의는 다음과 같음. control points가 4개 P_0, P_1, P_2, P_3 이라고 할 때,

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3, \quad t \in [0,1].$$

- 2차(Quadratic)인 경우 control points가 3개이고, 3차(Cubic)는 4개임.
- 데이터를 스무딩하기 위해서는 데이터 포인트를 어떻게 control points로 설정할지(또는 fitting할지)가 실무적으로 중요함.

구현

- 만약 4개의 포인트를 갖고 이를 "단 한 번의" 베지어 곡선으로 그릴 거라면, 위의 수식을 그대로 사용하면 됨.
- 여러 포인트가 있을 경우, **세그먼트별로** 구간을 나눈 뒤, 각 구간마다 베지어 곡선을 연결(피스와이즈)하여 스무딩함.
- 혹은 베지어가 아닌 **B-스플라인(B-spline)** 을 사용해 연속적으로 부드러운 곡선을 얻을 수도 있음(방법은 유사).

구현 예시 (Python)

```python
def cubic_bezier(p0, p1, p2, p3, num_samples=100):
    t_values = np.linspace(0, 1, num_samples)
    curve_points = []
    for t in t_values:
        b_t = (1 - t)**3 * p0 \
              + 3 * t * (1 - t)**2 * p1 \
              + 3 * (t**2) * (1 - t) * p2 \
              + (t**3) * p3
        curve_points.append(b_t)
    return np.array(curve_points)
```

3. Envelope (엔벌로프) 기반

개념

- 일반적으로는 "데이터의 최댓값(또는 최솟값)들을 이어 만든 곡선"을 의미하거나, 어떤 peak 부분을 감싸주는 상한/하한 곡선을 구해서 스무딩의 베이스로 사용하기도 함.
- 또는 **로우패스 필터**나 **이동평균(moving average)** 기법 등을 이용해서 빠르게 변동하는 고주파 성분을 제거한 뒤, 완만한 곡선만 남기고 그걸 'Envelope'로 부르는 경우도 있음.

구현

1. **상하 Envelope**
   - 데이터의 국소 최대값(local maxima)들만 뽑아서 스플라인(혹은 베지어 등)으로 연결한 것을 상(envelope)으로 사용하고, 국소 최소값들만 뽑아 연결해 하(envelope)로 사용하기도 함.
   - 실제 신호 분석 시엔 이 상·하부 Envelope로부터 진폭, 피크 시점 등을 뽑아서 특징을 추출함.
2. **Moving Average(이동평균) + Peak 검출**
   - Raw signal 을 window_size 길이로 평균 내서 점진적으로 부드러운 곡선을 만들면, 이 곡선이 Envelope처럼 동작하기도 함(단, 스파이크가 큰 경우엔 별도 처리가 필요).

구현 예시 (Python)

```
: def moving_average_envelope(signal, window_size=5):
    # 단순 이동평균으로 스무딩
    cumsum = np.cumsum(np.insert(signal, 0, 0))
    smoothed = (cumsum[window_size:] - cumsum[:-window_size]) / window_size
    return smoothed
```

4. Median 기반

개념

- 슬라이딩 윈도우 내 값들을 정렬 후 중앙값을 출력하는 비선형 필터

수식

- y[i]=median(x[i－⌊k/2⌋], …, x[i], …, x[i+⌊k/2⌋])

구현 예시 (Python)

```
def median_filter(signal, k):
    half = k // 2
    padded = np.pad(signal, half, mode='reflect')
    return np.array([np.median(padded[i:i+k]) for i in range(len(signal))])
```

5 Arithmetic Mean (산술평균) 기반

개념

- 슬라이딩 윈도우 내 값들의 산술 평균 출력

수식

$$y[i] = \frac{1}{k} \sum_{j=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} x[i+j]$$

구현 예시 (Python)

```
arithmetic_mean = uniform_filter(data, size=k, mode='constant')
```

6 Geometric Mean (기하평균) 기반

개념

- 슬라이딩 윈도우 내 값들의 기하평균을 출력

수식

$$y[i] = \left(\prod_{j=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} x[i+j] \right)^{\frac{1}{k}}$$

구현 예시 (Python)

```python
def geo_mean_func(window):
    return window.prod()**(1.0/len(window))
```